**Name** : Bhoomi Mangesh Naik
**Class** : D15C
**Roll No.** : 34
**Practical No.** : 7

**Aim :** Build an Artificial Neural Network (ANN) using Keras/TensorFlow.

## Theory :

## 1. Dataset Source

The dataset used for this experiment is obtained from Kaggle:

**Mall Customers Dataset**
https://www.kaggle.com/datasets/shwetabh123/mall-customers

## 2. Dataset Description

The Mall Customers dataset contains customer information collected from a shopping mall for market segmentation.

**Dataset Characteristics**

- **Total Records:** 200
- **Type:** Structured numerical dataset
- **Learning Type:** Unsupervised learning

**Features Used**

| Feature | Description |
|---|---|
| Annual Income (k$) | Customer's annual income |
| Spending Score (1–100) | Customer spending behavior |

There is **no target variable**, making the dataset suitable for clustering.

# 3. Mathematical Formulation

### 3.1 Artificial Neural Network (ANN)

An Artificial Neural Network consists of layers of interconnected neurons. Each neuron computes a weighted sum of inputs followed by an activation function.

$$\boxed{z = \sum_{i=1}^{n} w_i x_i + b}$$

Where:

- wiw_iwi = weights
- xix_ixi = input features
- bbb = bias
- zzz = linear combination

Output of neuron:

a=f(z)
Where:

- f(z)f(z)f(z) = activation function (ReLU, Sigmoid, Softmax)

### 3.2 Loss Function

The loss function measures the difference between actual and predicted output.

For binary classification:

$$\boxed{L = -\frac{1}{N} \sum [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]}$$

Where:

- yyy = actual value
- y^ = predicted value

---

3.3 Backpropagation

Weights are updated using gradient descent:

$$\boxed{w = w - \eta \frac{\partial L}{\partial w}}$$

Where:

- η = learning rate
- $\boxed{\frac{\partial L}{\partial w}}$ = gradient of loss

## 4. Algorithm Limitations

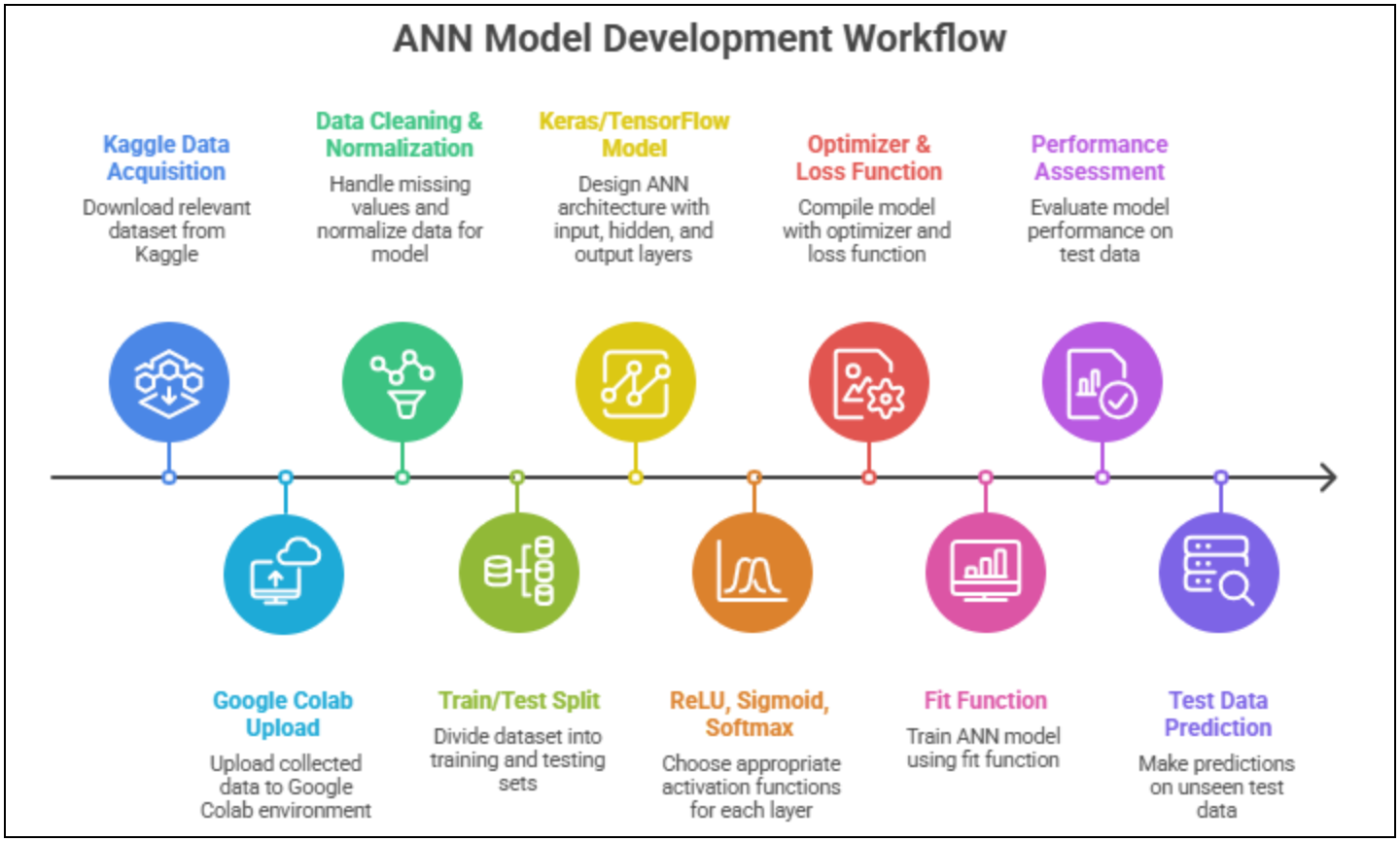
**ANN Limitations**

- Requires large amount of data for better performance
- Computationally expensive
- Can overfit if not properly regularized
- Requires careful tuning of hyperparameters
- Less interpretable compared to traditional models

## 5. Methodology / Workflow

- Dataset collection from Kaggle
- Data upload in Google Colab
- Data preprocessing (handling missing values, normalization)
- Splitting dataset into training and testing sets
- Designing ANN model using Keras/TensorFlow
- Adding input, hidden, and output layers
- Selecting activation functions (ReLU, Sigmoid, Softmax)
- Compiling model (optimizer, loss function)
- Training the model (fit function)
- Evaluating model performance
- Prediction on test data

**Workflow Diagram:**




## 6. Performance Analysis

- Evaluated using accuracy, loss, and validation metrics
- Training and validation curves help detect overfitting
- High accuracy and low loss indicate good model performance
- Confusion matrix can be used for classification evaluation

## 7. Hyperparameter Tuning

### ANN Hyperparameters

- Number of hidden layers
- Number of neurons per layer
- Learning rate
- Batch size
- Number of epochs
- Activation functions

**Impact:**

- Proper tuning improves model accuracy and generalization

- Prevents overfitting and underfitting
- Leads to better prediction performance

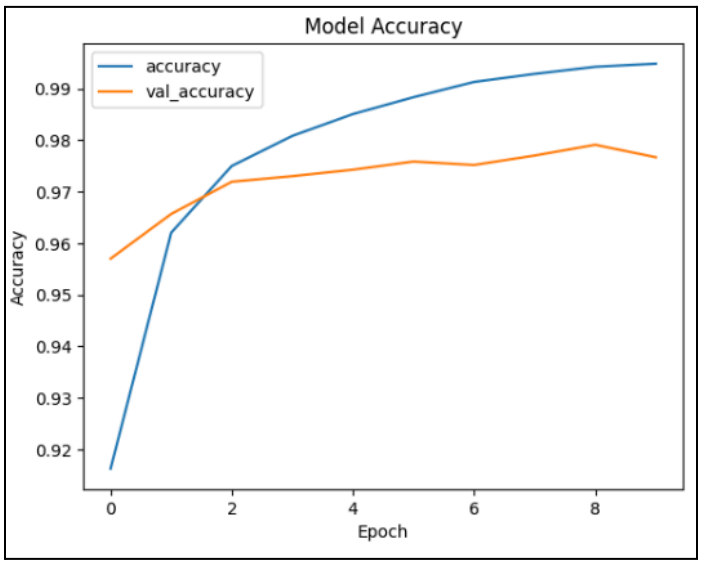
## Code and Output :

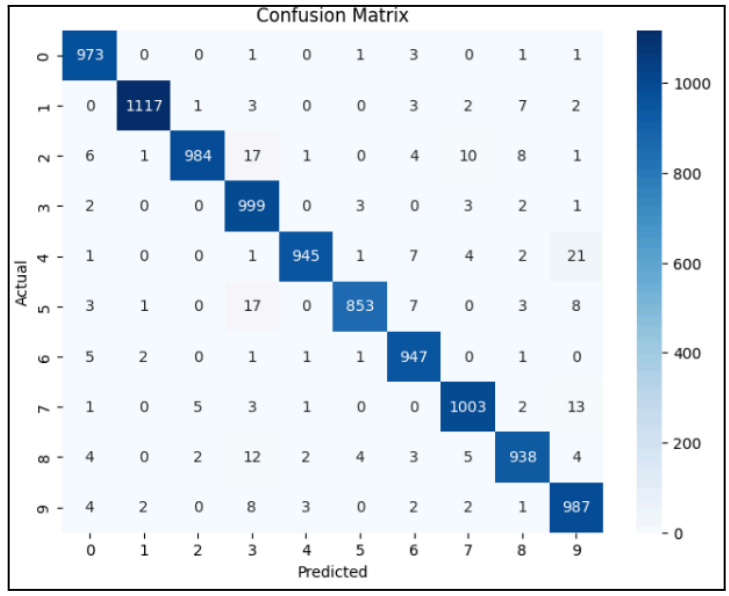
```
!pip install keras-tuner
from google.colab import files
import pandas as pd
# Upload dataset
uploaded = files.upload()
# Example (only if using CSV dataset)
# df = pd.read_csv("your_dataset.csv")
# print(df.head())
import tensorflow as tf
from tensorflow import keras
import keras_tuner as kt
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix
print("TensorFlow Version:", tf.__version__)
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
print("Initial Shape:", x_train.shape)
print("Missing Values: None (Image dataset)")
# Normalize
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
print("Data normalized")
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28,28)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
history = model.fit(
    x_train, y_train,
    epochs=10,
    validation_split=0.2
)
```

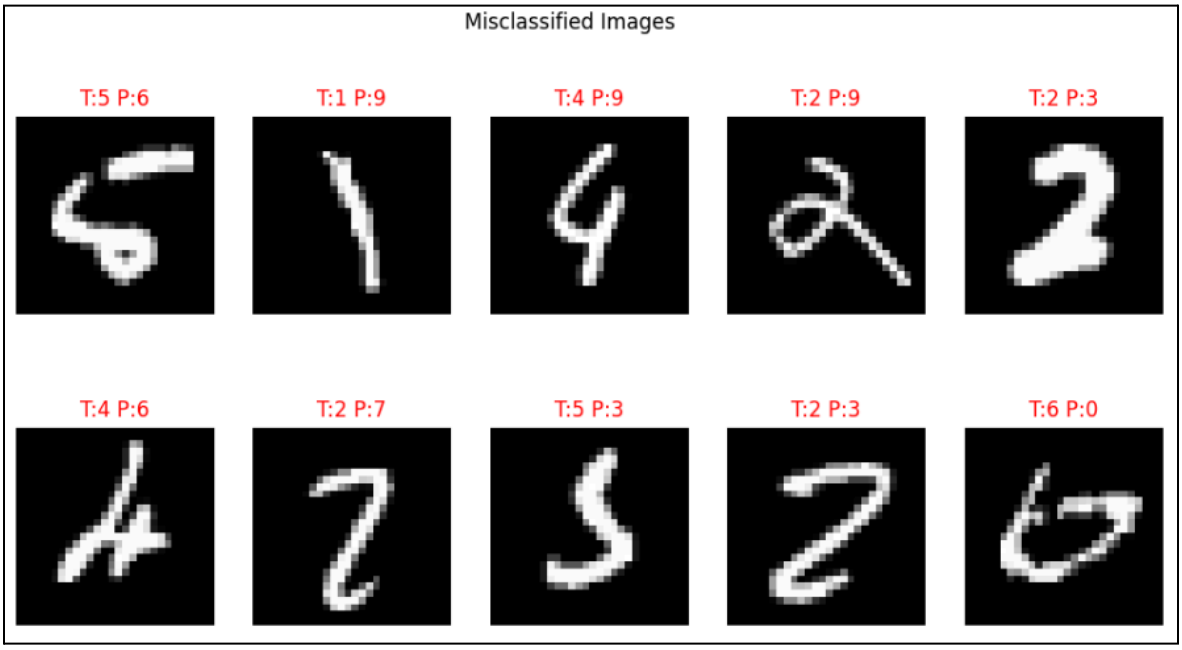
```
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_acc)
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.title("Model Accuracy")
plt.show()
def model_builder(hp):
    model = keras.Sequential()
    model.add(keras.layers.Flatten(input_shape=(28,28)))
    hp_units = hp.Int('units', min_value=32, max_value=512, step=32)
    model.add(keras.layers.Dense(units=hp_units, activation='relu'))
    model.add(keras.layers.Dense(10, activation='softmax'))
    hp_lr = hp.Choice('learning_rate', values=[1e-2, 1e-3, 1e-4])
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=hp_lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model
tuner = kt.Hyperband(
    model_builder,
    objective='val_accuracy',
    max_epochs=10,
    factor=3,
    directory='my_dir',
    project_name='ann_tuning'
)
tuner.search(
    x_train, y_train,
    epochs=10,
    validation_split=0.2
)
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_acc)
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
```

```
plt.title("Model Accuracy")
plt.show()
def model_builder(hp):
    model = keras.Sequential()
    model.add(keras.layers.Flatten(input_shape=(28,28)))
    hp_units = hp.Int('units', min_value=32, max_value=512, step=32)
    model.add(keras.layers.Dense(units=hp_units, activation='relu'))
    model.add(keras.layers.Dense(10, activation='softmax'))
    hp_lr = hp.Choice('learning_rate', values=[1e-2, 1e-3, 1e-4])
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=hp_lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model
tuner = kt.Hyperband(
    model_builder,
    objective='val_accuracy',
    max_epochs=10,
    factor=3,
    directory='my_dir',
    project_name='ann_tuning'
)
tuner.search(
    x_train, y_train,1
    epochs=10,
    validation_split=0.2
)
best_hps = tuner.get_best_hyperparameters(1)[0]
model = tuner.hypermodel.build(best_hps)
history = model.fit(
    x_train, y_train,
    epochs=10,
    validation_split=0.2
)
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)
conf_matrix = confusion_matrix(y_test, y_pred_classes)
plt.figure(figsize=(8,6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix")
plt.show()
errors = (y_pred_classes != y_test)
x_errors = x_test[errors]
y_errors = y_test[errors]
y_pred_errors = y_pred_classes[errors]
plt.figure(figsize=(12,6))
for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(x_errors[i], cmap='gray')
    plt.title(f"T:{y_errors[i]} P:{y_pred_errors[i]}", color='red')
    plt.axis('off')
plt.suptitle("Misclassified Images")
plt.show()
```



Google Colab Link for Code and Output : Link for Code and Output

## Conclusion :

In this experiment, an Artificial Neural Network (ANN) was implemented using TensorFlow and Keras on the MNIST dataset. The data was preprocessed and normalized to improve model performance. The model was trained using dense layers and evaluated using accuracy and loss metrics. Results showed that the ANN achieved high accuracy in classifying handwritten digits. Thus, ANN proved to be effective for solving image classification problems.